Introduction to Machine Learning
Problem Solutions: Introduction to Text

Prof. Sundeep Rangan

1. *BPE*: Suppose that you are given a sequence of characters

   [a, c, b, b, a, a, c, b, a, a, a]

   with an initial set of tokens {a, b, c}.

   (a) Use byte pair encoding to find a set of six tokens (three additional tokens from the initial set). In each step, when there are ties, select the leftmost frequent pair.

   (b) Encode the sequence with the tokens from part (a):

   [b,a,c,b,b,a,a,c,b]

Solution:

(a) Since we want to add three tokens, we have to run three steps of BPE.

Step 1. The bigrams are:

(a,c),(c,b),(b,b),(b,a),(a,a),(a,c),(c,b),(b,a),(a,a),(a,a)

The most frequent bigram is (a,a) which occurs 3 times. Note that the final a,a,a has two overlapping pairs of a,a and **both** are counted. So we merge to new token **aa**. New token set is

{a, b, c, aa}

Sequence becomes: [a, c, b, b, aa, c, b, aa, a].

Token set: {a,b,c,aa}.

Step 2. Encoding with the new tokens:

[a, c, b, b, aa, c, b, aa, a]

Bigrams are:

(a,c),(c,b),(b,b),(b,aa),(aa,c),(c,b),(b,aa),(aa,a)

Most frequent bigram is (c,b) which occurs 2 times. So we merge to new token **cb**. New token set is

{a, b, c, aa, cb}

Step 3. Encoding with the new tokens:

$$[\texttt{a, cb, b, aa, cb, aa, a}]$$

Bigrams are:

$$\texttt{(a,cb),(cb,b),(b,aa),(aa,cb),(cb,aa),(aa,a)}$$

All bigrams occur once. The leftmost is `(a,cb)`. So we merge to new token **acb**. New token set is

$$\{\texttt{a, b, c, aa, cb, acb}\}$$

(b) Using the above tokens, the encoded sequence is:

$$[\texttt{b,acb,b,aa,cb}]$$

2. *Embedding dimensions.* Suppose that the first layer of an network takes 1000 tokens out of vocabulary of 50000 possible tokens and embeds each token to a vector of dimension 512.

   (a) Find the number of parameters for this initial layer.

   (b) Suppose the input is a mini-batch of 100 samples or 1000 tokens each. What are the dimensions of the tensor at the input and output to this initial layer.

Solution:

(a) The embedding layer is represented by a matrix of size 50000×512. Hence the number of parameters is $50000 \cdot 512 = 25.6(10)^6$.

(b) For a mini-batch of 100 samples, each with 1000 tokens:

- **Input tensor:** Each token is an integer index in the vocabulary, so the input has shape $(100, 1000)$.
- **Output tensor:** Each token index is mapped to a 512-dimensional vector, so the output has shape $(100, 1000, 512)$.

3. *Text classification.* Suppose we want to train a simple network to classify text. We are given:

   - `doc_text`: A list of `n` text strings
   - `nclass`: An integer number of classes
   - `labels`: A vector of `n` integer labels from `0` to `nclass-1`
   - `emb = transform(text)`: A pre-trained function that maps list of text to embeddings. The output dimension is `emb_dim`.

   (a) Suppose you want to train a model, `mod`, to match embeddings to the class. What should the input and output dimensions for a batch of `nbatch` samples.

   (b) What loss function should the training use?

   (c) Given the above data, write numpy code to generate split the data into training and test tensors `Xtr,ytr` and `Xts,yts`. You can assume you have a function:

```
        Xtr,Xts,ytr,yts = train_test_split(X,y,test_frac)
```

Solution:

(a) Each text is mapped to an embedding of dimension `emb_dim`. Thus the input of a batch is of shape `(nbatch,emb_dim)`. The output is a class prediction among `nclass` classes, so the output dimension is of shape `(nbatch, nclass)`.

(b) Since this is a multi-class classification problem, the appropriate loss function is the **cross-entropy loss**, which compares the predicted probability distribution over classes with the true label.

(c) Code to generate training and test splits:

```
# Generate embeddings for all documents
X = transform(doc_text)   # shape (n, emb_dim)
y = np.array(labels)      # shape (n,)

# Split into training and test sets
Xtr, Xts, ytr, yts = train_test_split(X, y, test_frac=0.2)
```

4. *Document-query matching.* In this problem, we show how to use contrastive learning to train a neural network that matches documents in some corpus to text queries. Suppose each document and query are represented by text embeddings, `doc_emb` and `query_emb`, respectively, both of dimension `emb_dim`.

   (a) Complete the following PyTorch code for a simple neural network that takes batches of document and query embeddings, concatenates them, passes them through a two-layer fully connected neural network with ReLU activations to generate a scalar score `score` representing the match.

```
class MatchNet(nn.Module):
    def __init__(self, emb_dim, hidden_dim):
        super(MatchNet, self).__init__()

        # Create FC layers
        self.fc1 = ...
        self.fc2 = ...

    def forward(self, doc_emb, query_emb):
        ...
        score = ...
        return score
```

   You may use the following commands:

   - `C = torch.cat((A,B), dim=-1)`: Concatentates tensors `A` and `B` along the final dimension.

- `fc = nn.Linear(nin,nout`: Linear layer with `nin` input units and `nout` output units.
- `A = F.reul(B)`: ReLU activation of input `B`.

(b) You are given `doc_text`, `query_text`, each being lists of `n` text strings representing matching document and query pairs. You also have a function:

```
emb = transform(text)
```

that is some pre-trained transformer of lists of texts to their embeddings. Write code to generate:

- Generate embedding tensors `doc1` and `query1` of matching documents and query pairs (these are called positive samples)
- Generate embedding tensors `doc0` and `query0` of randomly document and query pairs (these are called negative samples)

You should generate `n` positive samples `10*n` negative samples.

You may use the function:

```
ind  = torch.randint(0, n, shape)
```

which generates a tensor of size `shape` where the entries are random integers from `0` to `n-1`.

Solution:

(a) One solution is as follows: Note that the first layer has `2*emb_dim` input units since it takes the concatenation of query and document embeddings. Also, the second layer has a single output since it is a scalar score.

```
class MatchNet(nn.Module):
    def __init__(self, emb_dim, hidden_dim):
        super(MatchNet, self).__init__()
        # Create FC layers
        self.fc1 = nn.Linear(2*emb_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, 1)

    def forward(self, doc_emb, query_emb):
        # Concatenate document and query embeddings along last dimension
        x = torch.cat([doc_emb, query_emb], dim=-1)
        # Pass through first layer + nonlinearity
        h = F.relu(self.fc1(x))
        # Pass through second layer to get scalar score
        score = self.fc2(h)
        return score
```

(b) The simplest way to do this is:

```
# Embed the positive samples
doc1 = transform(doc_text)
query1 = transform(query_text)
```

```
        # Randomly select negative samples
        doc_ind = torch.randint(0,n, 10*n)
        query_ind = torch.randint(0,n, 10*n)

        # Generate negative samples
        doc0 = doc1[doc_ind,:]
        query1 = query0[doc_ind,:]
```